

Friend Functions and Static Class Members

Friend Functions

What is a Friend Function?

A **friend function** is a function that is **declared inside a class but defined outside it**, and is given special permission to access the **private and protected members** of that class. It is declared using the `friend` keyword.

Normally, private members are only accessible from within the class. A friend function is an exception — it can access private data directly without being a member function.

```
class Sample {
    int a, b;
public:
    void setValue() { a = 25; b = 40; }
    friend float mean(Sample s);    // friend declaration
};

float mean(Sample s) {              // defined outside – but can access private a and b
    return float(s.a + s.b) / 2.0;
}

int main() {
    Sample x;
    x.setValue();
    cout << mean(x);    // called like a normal function, not on an object
}
```

Characteristics of Friend Functions

- Not in the scope of the class — it cannot be called using an object (e.g., `x.mean()` would be wrong)
- Does **not** have a `this` pointer
- Can be declared in either the `public` or `private` section without affecting its meaning
- Usually takes the class object as an argument to access its members
- Can be a friend of **more than one class** — useful for functions that need access to members of two different classes

Friend Class

An entire class can be declared as a friend, giving all its member functions access to the private members of another class:

```
class ClassA {
    friend class ClassB;    // all members of ClassB can access ClassA's private data
    int secret;
};
```

Static Data Members

A **static data member** is a member variable shared by all objects of a class. Unlike regular data members (where each object gets its own copy), there is **only one copy** of a static member, shared across all instances.

Properties of Static Data Members

- Initialised to **zero** when the first object is created
- Only **one copy** exists regardless of how many objects are created
- Visible only within the class but its **lifetime is the entire program**
- Must be **defined outside** the class after the class declaration

```
class Item {
    static int count;    // declaration inside class
    int number;
public:
    void getdata(int a) { number = a; count++; }
    void getcount() { cout << "Count: " << count; }
};

int Item::count;        // definition outside class – required

int main() {
    Item a, b, c;
    a.getcount();       // Count: 0
    b.getcount();       // Count: 0
    a.getdata(100); b.getdata(200); c.getdata(300);
    a.getcount();       // Count: 3 – shared across all objects
}
```

Static Member Functions

A **static member function** can only access other static members of the class. It can be called using the **class name** (without needing an object):

```
class Test {
    static int count;
public:
    static void showCount() {
        cout << "Count: " << count;
    }
};

int main() {
    Test::showCount();    // called on class, not on an object
}
```

Why use static members?

- Tracking how many objects of a class have been created
- Shared configuration values that apply to all objects
- Utility functions that logically belong to a class but don't need object state